An integrated method for reconstructing gene regulatory networks and pseudotime-based trajectories from single-cell RNA sequencing data

---

Abstract

Single-cell RNA sequencing (scRNA-seq) enables transcriptomic profiling at single-cell resolution, revealing cellular heterogeneity and continuous developmental transitions that are invisible to bulk approaches. Here we describe a complete, step-by-step computational pipeline for processing 10x Genomics Chromium scRNA-seq data — from raw FASTQ files through quality control, normalization, dimensionality reduction, batch correction, clustering, cell-type annotation, trajectory inference, and gene regulatory network reconstruction. All analyses are performed within a fully reproducible Docker environment (`mvergara19/scrnaseq_docker:latest`) containing R 4.5 and Python 3, including Seurat 5, monocle3, harmony, DESeq2, scanpy, scFates, and CellRanger 9.0.1. The protocol is applicable to any droplet-based scRNA-seq dataset and produces publication-ready figures together with a gene co-expression network amenable to functional enrichment analysis.

---

1 Introduction

The transcriptome of a cell reflects its functional state at a given moment in time. Bulk RNA sequencing averages this signal across millions of cells, masking rare populations and continuous developmental transitions. Single-cell RNA sequencing (scRNA-seq) circumvents this limitation by capturing the polyadenylated mRNA of each individual cell, generating a sparse count matrix in which rows represent genes and columns represent barcoded cells.

Key analytical challenges unique to scRNA-seq data include:

- **High dimensionality** — libraries typically measure 20,000–30,000 genes per cell, requiring dimensionality reduction before visualization or clustering.
- **Sparsity and dropout** — low-abundance transcripts are stochastically undetected, producing a high proportion of zero counts.
- **Ambient RNA contamination** — free RNA in the cell suspension is captured by empty droplets, inflating counts in certain barcodes.
- **Doublets** — occasionally two cells are co-encapsulated in a single droplet, producing artificially merged transcriptomes.

- **Batch effects** — technical variation between sequencing runs, donors, or library preparation batches obscures biological signal.
- **Trajectory inference** — cells undergoing differentiation exist along a continuum and must be ordered along a pseudotime axis to reveal gene expression dynamics.
- **Gene regulatory networks (GRNs)** — inferring directed regulatory interactions from co-expression patterns requires careful statistical modelling.

This protocol integrates three complementary frameworks: **Seurat** (R) for quality control, normalization, and clustering; **monocle3** (R) for graph-based trajectory inference and pseudotime ordering; and **scanpy/scFates** (Python) for diffusion-based pseudotime and gene dynamics modelling. All software is pre-installed in a single Docker image, ensuring full reproducibility across operating systems and institutions.

---

2 Materials

2.1 Computational environment

All analyses described in this protocol are executed inside a Docker container. No local R or Python installation is required beyond Docker itself.

Minimum hardware requirements:

Resource	Recommended
RAM	32 GB (64 GB for datasets > 50,000 cells)
Storage	50 GB free disk space
CPU	8 cores

Docker image:

```
docker pull mvergara19/scrnaseq_docker:latest
```

Start an interactive Bash session:

```
docker run --rm -it -v $(pwd):/workspace mvergara19/scrnaseq_docker:latest bash
```

All downstream commands in this protocol are intended to be run from within this interactive session. The `/workspace` directory inside the container is a live mount of your current local folder.

2.2 Software versions

Tool	Version	Role
R	4.5	Primary analysis environment
Seurat	5.4.0	QC, normalization, clustering
SeuratDisk	0.0.0.9021	h5seurat / h5ad I/O
SeuratWrappers	0.4.0	External method integrations
monocle3	1.4.26	Trajectory inference
harmony	1.2.4	Batch correction
DESeq2	1.50.2	Differential expression

Tool	Version	Role
DoubletFinder	2.0.6	Doublet detection
scater	1.38.1	QC visualization
zellkonverter	1.20.1	SCE <-> AnnData conversion
CellRanger	9.0.1	Read alignment and UMI counting
CellBender	0.3.0	Ambient RNA removal
scanpy	1.12	Python single-cell toolkit
scFates	1.2.3	Python trajectory analysis
palantir	1.4.4	Diffusion pseudotime

2.3 Input data

This protocol requires either:

1. Raw FASTQ files from a 10x Genomics Chromium 3' or 5' library, together with a reference transcriptome compatible with CellRanger (human GRCh38, mouse mm10, or a custom genome).
2. A pre-computed filtered feature-barcode matrix (output of CellRanger) in the standard MEX or HDF5 format.

All data files should be placed in the `workspace/` subdirectory on the host machine, which is mounted at `/workspace` inside the container.

---

3 Methods

3.1 Read alignment and UMI counting with CellRanger

CellRanger processes raw FASTQ files, aligns reads to a reference genome using STAR, performs barcode correction, and collapses unique molecular identifiers (UMIs) to produce a filtered feature-barcode count matrix.

```
cellranger count \
    --id=sample_id \
    --fastqs=/workspace/fastqs \
    --sample=sample_name \
    --transcriptome=/workspace/refdata
```

The primary output is located at `sample_id/outs/filtered_feature_bc_matrix/` and contains three compressed files: `barcodes.tsv.gz`, `features.tsv.gz`, and `matrix.mtx.gz`. These three files together constitute the count matrix in Market Exchange Format (MEX) and serve as input for all downstream R analyses.

> **Note:** CellRanger also produces a `raw_feature_bc_matrix/` directory containing all detected barcodes including empty droplets. This unfiltered matrix is required as input for CellBender (Section 3.2).

3.2 Ambient RNA removal with CellBender

Empty droplets in a 10x Genomics experiment contain ambient RNA from lysed cells. CellBender uses a deep generative model to distinguish genuine cell-containing droplets from empty ones and to estimate and subtract the ambient RNA signal from each cell's count profile.

```
cellbender remove-background \
    --input  /workspace/raw_feature_bc_matrix.h5 \
    --output /workspace/cellbender_output.h5
```

The corrected count matrix (`cellbender_output_filtered.h5`) is recommended as input for all downstream analyses, as it reduces false-positive marker gene calls caused by ambient contamination.

3.3 Project initialisation in R

All R analyses begin by loading the required packages and setting global parameters. The following libraries must be loaded at the start of every session:

```
library(Seurat)
library(SeuratDisk)
library(SeuratWrappers)
library(harmony)
library(monocle3)
library(DESeq2)
library(SingleCellExperiment)
library(zellkonverter)
library(scater)
library(DoubletFinder)
library(ggplot2)
library(patchwork)
library(cowplot)
library(dplyr)
library(tibble)
library(Matrix)
library(igraph)
library(clustree)
```

Key parameters that govern the analysis are defined once and reused throughout:

```
set.seed(42)

DATA_DIR    <- "/workspace/data"
OUTPUT_DIR  <- "/workspace/output"

MIN_FEATURES <- 200    # minimum genes detected per cell
MT_THRESHOLD <- 20     # maximum % mitochondrial counts
N_DIMS       <- 30     # number of PCA dimensions
RESOLUTION   <- 0.5    # Leiden clustering resolution
```

3.4 Data import

The CellBender-corrected HDF5 file is loaded directly into a Seurat object:

```
seurat <- LoadH5Seurat("/workspace/cellbender_output_filtered.h5")
```

Alternatively, when starting from a standard CellRanger MEX output:

```
counts <- Read10X(data.dir = file.path(DATA_DIR, "filtered_feature_bc_matrix"))
seurat <- CreateSeuratObject(
    counts       = counts,
    project      = "scRNASeq",
    min.cells    = 3,
    min.features = MIN_FEATURES
)
```

3.5 Quality control

3.5.1 Computing per-cell QC metrics

Three fundamental QC metrics are computed for each cell:

1. **Number of detected genes** (`nFeature_RNA`) — proxy for sequencing depth; very low values indicate empty droplets, very high values may indicate doublets.
2. **Total UMI count** (`nCount_RNA`) — total number of transcripts detected.
3. **Percentage of mitochondrial reads** (`percent.mt`) — elevated values indicate damaged or dying cells whose cytoplasmic mRNA has leaked out, leaving mitochondrial transcripts disproportionately represented.

```
seurat[["percent.mt"]] <- PercentageFeatureSet(seurat, pattern = "^MT-")
seurat[["percent.rb"]] <- PercentageFeatureSet(seurat, pattern = "^RP[SL]")
```

3.5.2 Visualising QC distributions

Violin plots allow visual inspection of the metric distributions and guide threshold selection (Figure 1):

```
VlnPlot(
    seurat,
    features = c("nFeature_RNA", "nCount_RNA", "percent.mt"),
    ncol = 3,
    pt.size = 0.1
)
```

Scatter plots reveal the relationship between sequencing depth and gene detection, and help identify outlier cells (Figure 2):

```
FeatureScatter(seurat, feature1 = "nCount_RNA", feature2 = "nFeature_RNA")
FeatureScatter(seurat, feature1 = "nCount_RNA", feature2 = "percent.mt")
```

3.5.3 Filtering low-quality cells

Cells failing any of the following criteria are removed:

```
seurat <- subset(
    seurat,
    subset = nFeature_RNA > 200   &
             nFeature_RNA < 6000  &
             percent.mt   < 20
)
```

Thresholds should be adjusted based on the observed distributions. A common heuristic is to exclude cells more than three median absolute deviations (MADs) from the median of each metric.

3.6 Doublet detection

Doublets are identified using DoubletFinder, which simulates artificial doublets by randomly averaging pairs of cell profiles and identifies real cells whose transcriptomes closely resemble those artificial doublets. The expected doublet rate for 10x Genomics libraries is approximately 0.8% per 1,000 cells recovered.

```
# Estimate optimal neighbourhood size (pK) via parameter sweep
sweep_res  <- paramSweep(seurat, PCs = 1:N_DIMS, sct = FALSE)
sweep_stats <- summarizeSweep(sweep_res, GT = FALSE)
bcmvn      <- find.pK(sweep_stats)
pK_opt     <- as.numeric(as.character(bcmvn$pK[which.max(bcmvn$BCmetric)]))

# Expected number of doublets
nExp <- round(0.075 * ncol(seurat))

seurat <- doubletFinder(
    seurat,
    PCs  = 1:N_DIMS,
    pN   = 0.25,
    pK   = pK_opt,
    nExp = nExp,
    sct  = FALSE
)

seurat <- subset(seurat, subset = DF.classifications_0.25 == "Singlet")
```

3.7 Normalisation and feature selection

3.7.1 Log-normalisation

Raw counts are normalised to a library size of 10,000 UMIs per cell and log-transformed to stabilise variance across the dynamic range:

```
seurat <- NormalizeData(
    seurat,
    normalization.method = "LogNormalize",
    scale.factor = 10000
)
```

3.7.2 Identification of highly variable genes

Downstream analyses focus on the 2,000 most variable genes, selected using the variance-stabilising transformation (VST) method:

```
seurat <- FindVariableFeatures(
    seurat,
    selection.method = "vst",
    nfeatures = 2000
)
```

3.7.3 Data scaling

Gene expression is centred and scaled across cells prior to PCA, so that highly expressed genes do not dominate the principal components:

```
seurat <- ScaleData(seurat, features = rownames(seurat))
```

3.8 Dimensionality reduction

3.8.1 Principal component analysis

PCA is applied to the scaled expression matrix of highly variable genes. The number of informative PCs is estimated from an elbow plot, where the y-axis (standard deviation explained) plateaus (Figure 3):

```
seurat <- RunPCA(seurat, features = VariableFeatures(seurat), npcs = 50)
ElbowPlot(seurat, ndims = 50)
```

3.8.2 Batch correction with Harmony

When the dataset contains cells from multiple samples, donors, or sequencing runs, Harmony iteratively adjusts the PCA embedding to remove batch-specific shifts while preserving biological variation:

```
seurat <- RunHarmony(
    seurat,
    group.by.vars = "sample",
    reduction     = "pca",
    dims.use      = 1:N_DIMS
)
```

The resulting `harmony` reduction replaces `pca` as input for graph construction and UMAP in all subsequent steps.

3.8.3 Uniform manifold approximation and projection

UMAP reduces the high-dimensional PCA or Harmony embedding to two dimensions for visualisation, preserving both local and global structure:

```
seurat <- RunUMAP(seurat, dims = 1:N_DIMS, reduction = "pca")
```

3.9 Clustering

3.9.1 Shared nearest-neighbour graph construction

A k-nearest-neighbour (kNN) graph is first constructed in PCA space, then refined into a shared nearest-neighbour (SNN) graph by retaining edges where the two cells share many neighbours:

```
seurat <- FindNeighbors(seurat, dims = 1:N_DIMS)
```

3.9.2 Resolution selection with clustree

The Leiden community detection algorithm is applied over a range of resolutions. The `clustree` package visualises how clusters split or merge as resolution increases, facilitating selection of a stable partition (Figure 4):

```
seurat <- FindClusters(seurat, resolution = seq(0.1, 1.2, by = 0.1))
clustree(seurat, prefix = "RNA_snn_res.")
```

3.9.3 Final clustering

After visual inspection of the clustree output, the chosen resolution is applied:

```
seurat <- FindClusters(seurat, resolution = 0.5)
DimPlot(seurat, reduction = "umap", label = TRUE, pt.size = 0.5)
```

3.10 Cell-type annotation

3.10.1 Marker gene identification

For each cluster, genes that are significantly overexpressed relative to all other clusters are identified using a Wilcoxon rank-sum test:

```
markers <- FindAllMarkers(
    seurat,
    only.pos       = TRUE,
    min.pct        = 0.25,
    logfc.threshold = 0.25
)
```

Results are filtered and ranked by log2 fold change to produce a shortlist of candidate marker genes per cluster.

3.10.2 Visualisation of marker expression

Dot plots display the average expression and percentage of expressing cells for selected markers across all clusters simultaneously (Figure 5), while feature plots project expression values onto the UMAP embedding to confirm cluster-specific enrichment (Figure 6):

```
DotPlot(seurat, features = top_markers) + RotatedAxis()
FeaturePlot(seurat, features = c("CD3D", "CD14", "MS4A1"), ncol = 3)
```

3.10.3 Assignment of cell-type labels

After cross-referencing marker genes against curated databases (e.g., CellMarker, PanglaoDB) or published signatures, clusters are annotated:

```
new_ids <- c("0" = "CD4 T cells", "1" = "CD14 Monocytes",
             "2" = "B cells",     "3" = "CD8 T cells",
             "4" = "NK cells",    "5" = "FCGR3A Monocytes")
seurat <- RenameIdents(seurat, new_ids)
seurat$cell_type <- Idents(seurat)
```

3.11 Differential expression analysis

Pseudo-bulk differential expression is performed with DESeq2, which models count data with a negative binomial distribution and applies empirical Bayes shrinkage of fold change estimates. Cells are first aggregated per sample and cell type to produce pseudo-bulk profiles, avoiding the inflation of statistical power that arises from treating each cell as an independent replicate:

```
sce    <- as.SingleCellExperiment(seurat)
pseudo <- aggregateAcrossCells(sce, ids = colData(sce)[, c("cell_type", "sample")])

dds <- DESeqDataSet(pseudo, design = ~ cell_type)
dds <- DESeq(dds)
res <- results(dds, contrast = c("cell_type", "CD8 T cells", "CD4 T cells"))
```

Results are filtered at a false discovery rate of 5% and ranked by log2 fold change for downstream interpretation.

3.12 Trajectory inference with monocle3

monocle3 represents cell fate decisions as a principal graph learned directly from the low-dimensional embedding. Cells are ordered along the graph to assign a pseudotime value reflecting their relative developmental progression.

3.12.1 Conversion from Seurat to CellDataSet

```
cds <- as.cell_data_set(seurat)
cds <- preprocess_cds(cds, num_dim = N_DIMS)
cds <- reduce_dimension(cds)
cds <- cluster_cells(cds)
```

3.12.2 Learning the principal graph

```
cds <- learn_graph(cds)
```

The graph is visualised overlaid on the UMAP embedding (Figure 7). Branch points correspond to fate decision events; leaf nodes represent terminal cell states.

3.12.3 Pseudotime ordering

A root cell or population representing the earliest developmental state is selected, and pseudotime is propagated along the graph:

```r
cds    <- order_cells(cds, root_cells = root_cells)
plot_cells(cds, color_cells_by = "pseudotime",
           label_cell_groups = FALSE)
```

3.12.4 Identification of pseudotime-dependent genes

Moran's I spatial autocorrelation statistic is used to identify genes whose expression changes significantly along the principal graph:

```r
pr_test <- graph_test(cds, neighbor_graph = "principal_graph", cores = 4)
sig_genes <- rownames(subset(pr_test, q_value < 0.05))
```

3.13 Trajectory analysis with scFates (Python)

scFates provides complementary trajectory analysis based on diffusion maps and principal trees, and fits gene dynamics curves along the inferred trajectory.

```python
import scanpy as sc
import scFates as scf
import palantir

# Standard preprocessing
adata = sc.read_h5ad("/workspace/data.h5ad")
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, n_top_genes=2000)
sc.pp.pca(adata)
sc.pp.neighbors(adata, n_neighbors=15, n_pcs=30)
sc.tl.umap(adata)

# Diffusion maps for pseudotime
palantir.preprocess.run_diffusion_maps(adata)
palantir.preprocess.determine_multiscale_space(adata)

# Principal tree and pseudotime
scf.tl.tree(adata, method="ppt", Nodes=10, use_rep="X_diffmap")
scf.tl.pseudotime(adata, n_jobs=4)

# Gene dynamics along trajectory
scf.tl.test_association(adata, n_jobs=4)
scf.tl.fit(adata, n_jobs=4)

adata.write_h5ad("/workspace/trajectory_results.h5ad")
```

3.14 Gene co-expression network reconstruction

A Pearson correlation network is constructed from the normalised expression matrix of highly variable genes, following the approach described by Windram et al. (2018) for bulk RNA-seq data and here adapted for single-cell data.

3.14.1 Construction of the correlation matrix

The per-cell normalised expression values of the 2,000 highly variable genes are extracted and all pairwise Pearson correlation coefficients are computed:

```
expr <- as.matrix(GetAssayData(seurat, slot = "data")[VariableFeatures(seurat), ])
cor_mat <- cor(t(expr), method = "pearson")
```

The distribution of correlations is examined to select a stringent threshold (Figure 8). A threshold of |r| 0.7 is recommended as a starting point, balancing network density against specificity.

3.14.2 Network construction and filtering

An adjacency matrix is derived by retaining only gene pairs with |r| above the threshold:

```
adj <- abs(cor_mat) >= 0.7
diag(adj) <- FALSE
g <- graph_from_adjacency_matrix(adj, mode = "undirected", diag = FALSE)
g <- simplify(g)
```

3.14.3 Network statistics

Summary statistics characterising the network topology are computed and tabulated (Table 2):

```
data.frame(
    Metric = c("Nodes", "Edges", "Density",
               "Average degree", "Clustering coefficient"),
    Value  = c(vcount(g), ecount(g),
               round(edge_density(g), 4),
               round(mean(degree(g)), 2),
               round(transitivity(g), 4))
)
```

Scale-free networks — characteristic of biological regulatory networks — display a power-law degree distribution, where a small number of highly connected hub genes regulate many others.

3.14.4 Community detection

The Louvain algorithm partitions the network into modules — groups of genes more densely connected to each other than to the rest of the network:

```
communities <- cluster_louvain(g)
V(g)$module <- membership(communities)
```

Each module may represent a co-regulated gene programme associated with a specific cellular process or lineage.

3.14.5 Network visualisation

The largest connected component is visualised using a force-directed layout (Figure 9), with node size proportional to degree and node colour indicating module membership:

```
gcc <- induced_subgraph(g, which(components(g)$membership == 1))
plot(gcc,
```

```
     layout       = layout_with_fr(gcc),
     vertex.color = rainbow(max(V(gcc)$module))[V(gcc)$module],
     vertex.size  = log1p(degree(gcc)) * 2,
     vertex.label = NA,
     edge.color   = "grey80",
     edge.width   = 0.3)
```

3.14.6 Export for Cytoscape

The edge list and node attribute table are exported in CSV format for import into Cytoscape, where further visual refinement and functional enrichment analysis (e.g., with the ClueGO plugin) can be performed:

```
write.csv(as_data_frame(g, what = "edges"),
          "/workspace/output/network_edges.csv", row.names = FALSE)

write.csv(data.frame(gene   = V(g)$name,
                     degree = degree(g),
                     module = V(g)$module),
          "/workspace/output/network_nodes.csv", row.names = FALSE)
```

3.15 Integration of pseudotime and network modules

To assess temporal coordination of co-expression modules, the pseudotime at which each gene reaches its peak expression is computed as a weighted mean of cell pseudotime values, weighted by the normalised expression of the gene in each cell. Genes assigned to the same network module with overlapping pseudotime distributions are candidates for co-regulated gene programmes active during specific developmental windows (Figure 10).

3.16 Saving results

```
# Seurat object
saveRDS(seurat, "/workspace/output/seurat_final.rds")

# AnnData for Python interoperability
writeH5AD(as.SingleCellExperiment(seurat),
          "/workspace/output/seurat_final.h5ad")

# Differential expression results
write.csv(res, "/workspace/output/DESeq2_results.csv")

# Co-expression network
saveRDS(g, "/workspace/output/coexpression_network.rds")
```

---

4 Notes

1. **Cell Ranger reference genomes** are available for download from the 10x Genomics support website. Pre-built references for human (GRCh38) and mouse (GRCm38/mm10) are

recommended for standard experiments.

2. **CellBender GPU acceleration** — CellBender supports CUDA-enabled GPUs. To use GPU acceleration, the Docker container must be launched with `--gpus all` and the host must have the NVIDIA Container Toolkit installed.

3. **Harmony vs. other integration methods** — Harmony is recommended for datasets with moderate batch effects. For more severe technical variation or cross-species integration, Seurat's RPCA or scVI (a deep learning method available through SeuratWrappers) may yield better results.

4. **monocle3 root selection** — The choice of root cell strongly influences pseudotime assignments. Biological knowledge of the earliest cell state in the system should guide this selection. Transcription factor expression, differentiation markers, or RNA velocity can corroborate the assignment.

5. **Correlation thresholding** — The |r| 0.7 threshold is a heuristic. For datasets with fewer cells or higher dropout rates, a lower threshold (|r| 0.5) may be necessary. The choice of threshold should be validated by inspecting the degree distribution and comparing hub gene identity against prior literature.

6. **Reproducibility** — The `set.seed(42)` call at the beginning of the script ensures that stochastic steps (UMAP, Louvain, DoubletFinder) produce identical results across runs. The Docker image pins all software to exact versions, preventing dependency drift.

References

1. Hao Y, *et al.* (2024). Dictionary learning for integrative, multimodal and scalable single-cell analysis. *Nature Biotechnology* **42**, 293–304.

2. Cao J, *et al.* (2019). The single-cell transcriptional landscape of mammalian organogenesis. *Nature* **566**, 496–502.

3. Korsunsky I, *et al.* (2019). Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods* **16**, 1289–1296.

4. Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* **15**, 550.

5. Wolf FA, Angerer P, Theis FJ (2018). SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology* **19**, 15.

6. Fleming SJ, *et al.* (2023). Unsupervised removal of systematic background noise from droplet-based single-cell experiments using CellBender. *Nature Methods* **20**, 1323–1335.

7. McGinnis CS, Murrow LM, Gartner ZJ (2019). DoubletFinder: doublet detection in single-cell RNA sequencing data using artificial nearest neighbors. *Cell Systems* **8**, 329–337.

8. Sikkema L, *et al.* (2023). An integrated cell atlas of the lung in health and disease. *Nature Medicine* **29**, 1563–1577.

9. Windram O, *et al.* (2018). Step-by-step construction of gene co-expression networks from high-throughput *Arabidopsis* RNA sequencing data. In: *Plant Transcription Factors*, Methods in Molecular Biology vol. 1830, pp. 291–322. Humana Press, New York.